WaitCost — How Our Project & Data Work

A plain-English guide for the team: what we built, how the data flows, and the choices we made.

USAII Global AI Hackathon 2026 · Challenge 6A · calibrated for Los Angeles (CA-600)

How to read this guide: it goes from simple to detailed. Sections 1–3 are the plain overview anyone can follow. Sections 4–6 show where the data comes from. Section 7 is the deep, technical version — skip it unless you want it.

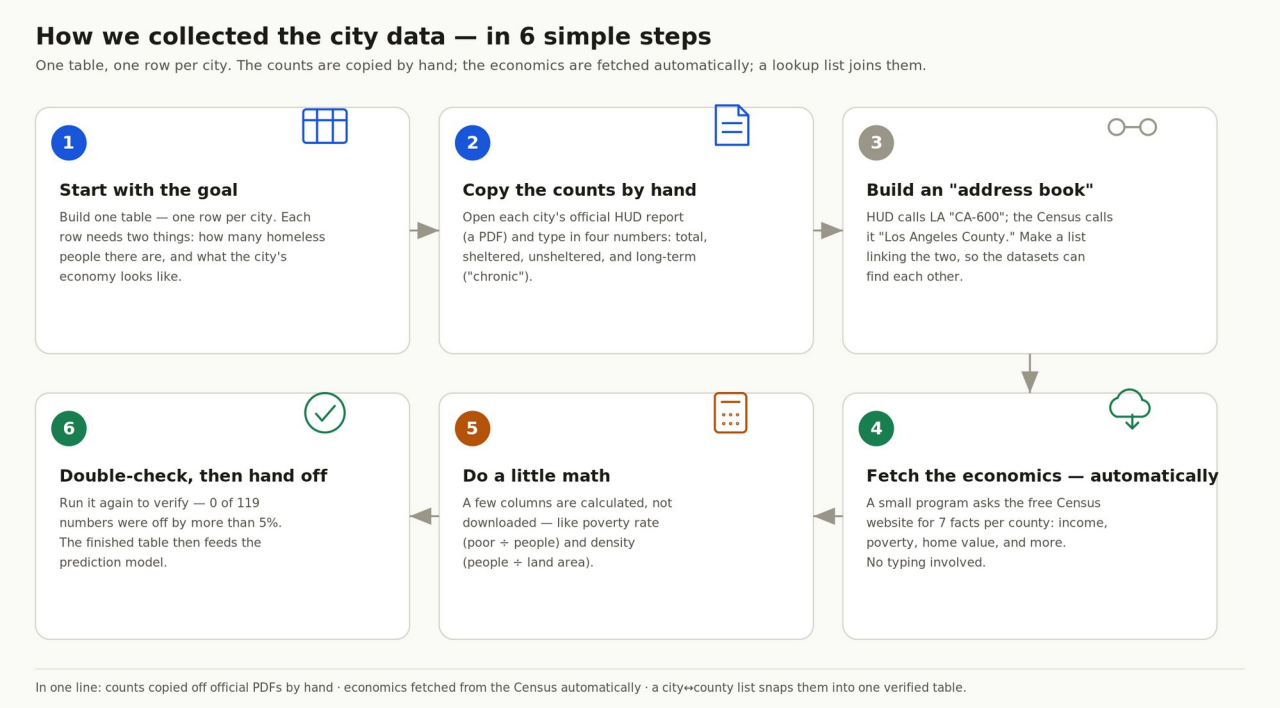
1. What WaitCost does

WaitCost helps a city budget office answer one hard question: if we delay funding a program to reduce homelessness, what does that delay cost us later?

You type a plain question — for example, “What if we wait 3 years on a $15 million program?” — and the tool gives a clear, sourced answer in dollars, with an honest range, plus a short written recommendation. It never decides the budget itself; a person always makes the final call. Everything is built on free, public data, and the whole thing runs offline on a laptop.

2. The whole journey in plain terms

Before the details, here is the entire data process as six simple steps.



Set the goal, copy the counts, link the sources, fetch the economics, do the math, verify.

In short: the homeless counts are copied by hand from official PDF reports, the economic numbers are pulled automatically from the Census, and a simple lookup list links the two so they match up correctly.

3. How the AI actually works

People often ask “where is the AI?” This picture answers it. Read it top to bottom.

The AI handles language; trusted Python code owns every number; the biggest decision pauses for a human.

The brain. A small AI model called Gemma — running offline on the laptop — reads your question and decides which tool is needed.

The sandbox. The actual maths is done by ordinary, trusted Python code: a simulator, the learned model, public-data lookups, and comparison metrics. The AI does not do the calculations.

The voice. The same Gemma model then explains the result in plain English.

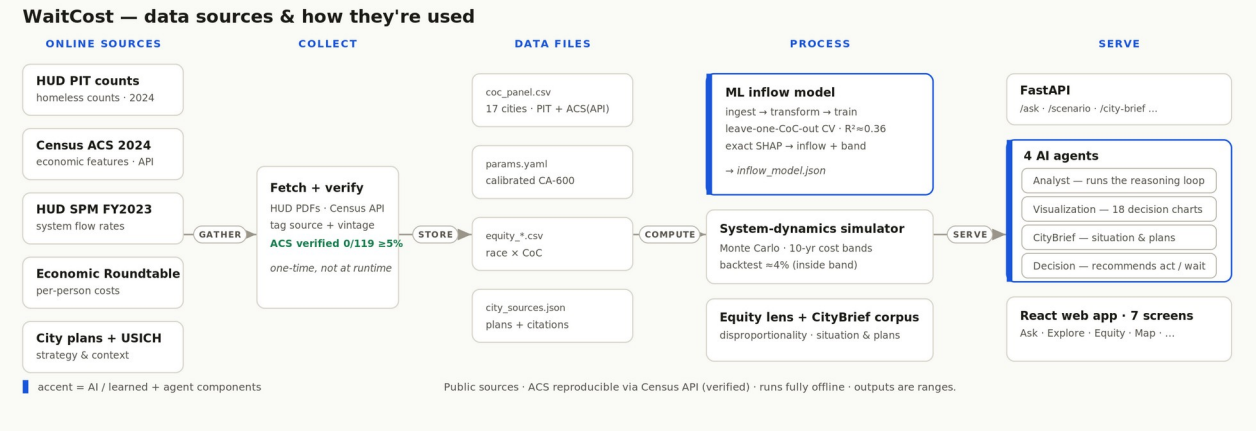
The output. Out comes the answer and a one-page decision brief — and every number in it comes from the code, never invented by the AI.

The human gate. Recommending actual spending is “Tier-2,” and the tool deliberately stops and asks a person there.

Why we built it this way: the AI is excellent at language — understanding the question and writing the explanation — but we don’t trust it with numbers. So the code owns every figure, and a human owns the final money decision.

4. The data, big picture

This is the data “plumbing,” from left to right — from public websites all the way to the app.



Online sources → collect & verify once → tidy data files → processed by the model & simulator → served to the app.

Online sources. The public websites where the data lives (HUD, Census, and others).

Collect. We gather it once and verify it — the Census numbers are pulled live from the official API and checked.

Data files. It’s saved into a few tidy files inside the project.

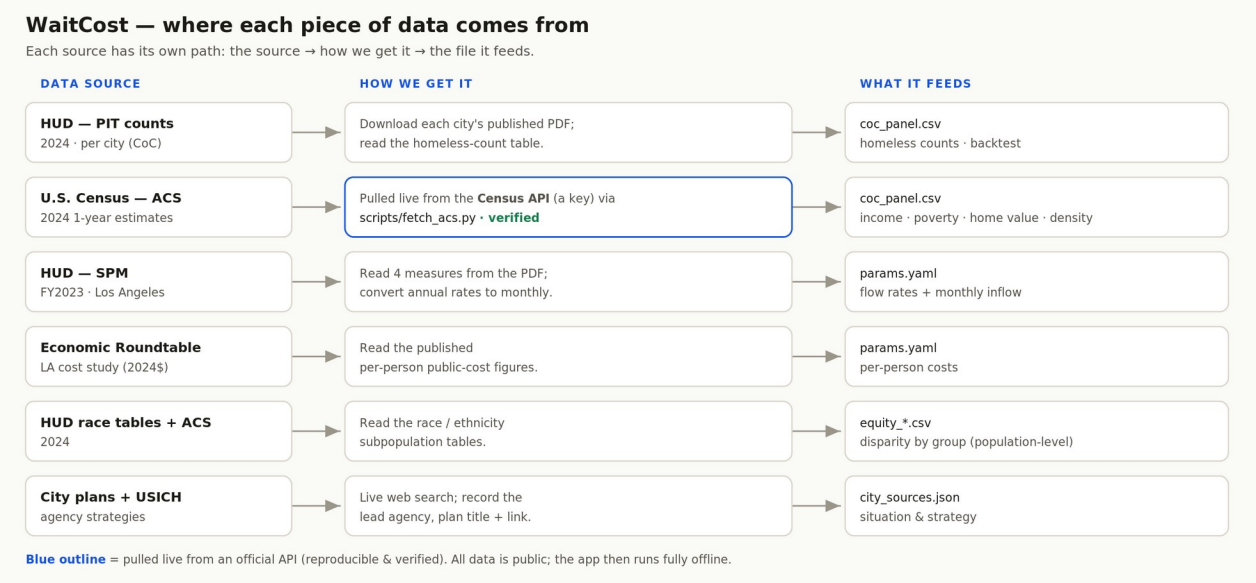
Process. Programs turn those files into the learned model, the simulator, the fairness lens, and the city write-ups.

Serve. The results are delivered through an API and four “agents” to the web app.

The point: we collect and verify the data once, then everything runs from local files — fast, repeatable, and fully offline.

5. Where each piece of data comes from

Same idea, but one row per source, so you can see each path: the source, how we get it, and the file it feeds. The blue-outlined row is pulled live from an official API and verified.



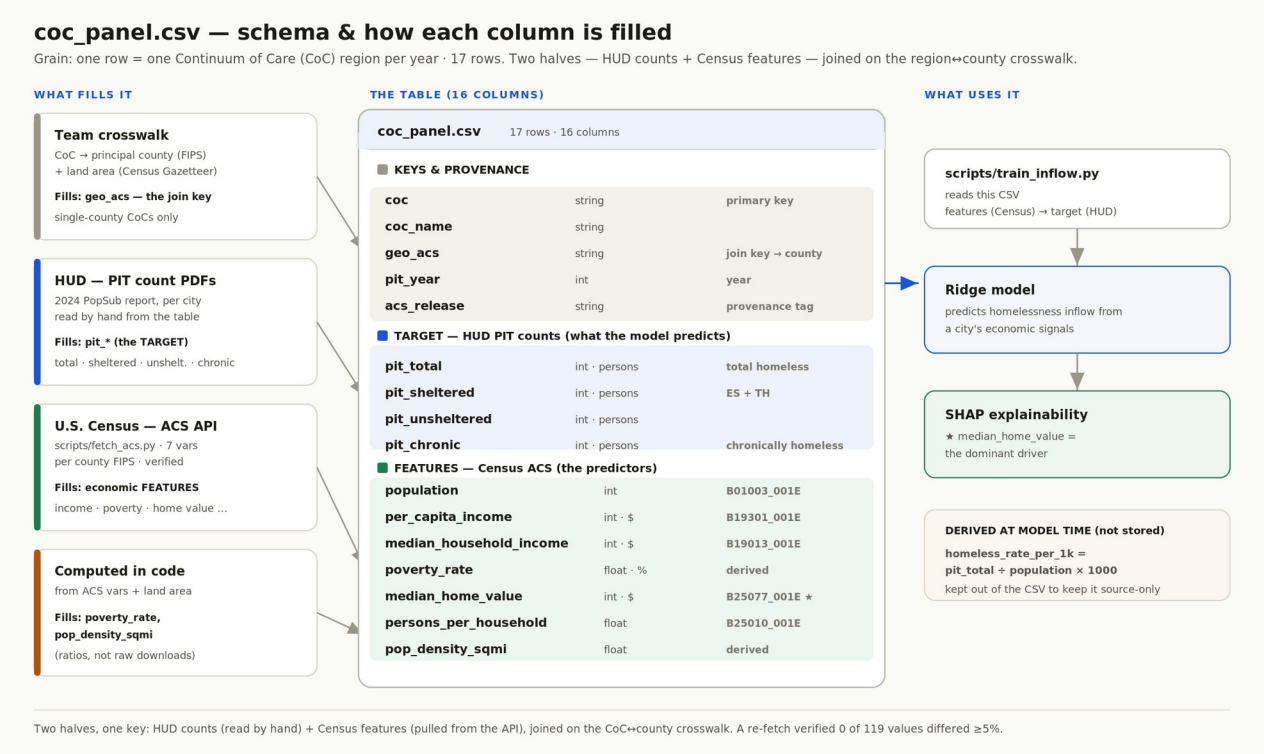
Each source has its own path: source → how we get it → the file it feeds.

We use these public sources, chosen because each is official, free, and updated regularly:

Source	What it gives us	Why we chose it
HUD Point-in-Time counts	How many people are homeless in each city, and their situation	The official U.S. government count — the most authoritative number available.
U.S. Census (ACS)	A city's economy: income, poverty, home prices, population	Free, reliable, updated every year; explains why homelessness differs city to city.
HUD System Performance Measures	How people move in and out of homelessness over time	Official data on the "flow," which our forward-looking simulation needs.
Economic Roundtable cost study	What one person's homelessness costs the public per month	A published Los Angeles study, so our dollar figures rest on real research, not guesses.

6. What’s inside our main dataset

All of this lands in one spreadsheet — one row per city, 17 cities in total. The picture shows what each column is and where it comes from.

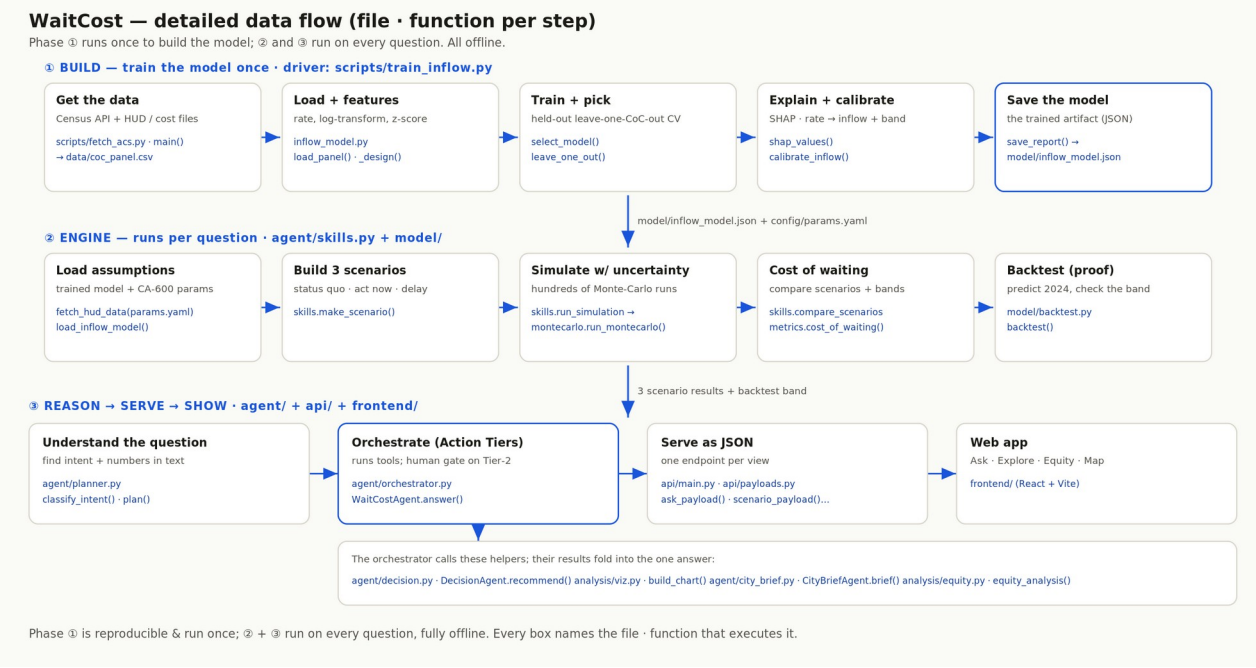


Two halves: what we predict (HUD counts) and the clues we use (Census economics).

Think of it in two halves: the “target” columns are what we are trying to understand (how many people are homeless), and the “feature” columns are the clues we use to explain it (income, poverty, home prices). A separate program learns the link between the clues and the outcome — and it found that housing cost is the biggest driver.

7. Under the hood — the detailed pipeline (for the curious)

This section is for anyone who wants the full technical picture. You can skip it if you just need the overview above. It names the exact file and function behind every step.



Three phases: build the model once · run the engine on every question · reason, serve and show.

Phase 1 — Build. Train the model one time: fetch the Census data, build features, train and honestly test it, explain it with SHAP, and save the trained model to a file.

Phase 2 — Engine. Runs on every question: load the assumptions, build the “act now / wait” scenarios, simulate years ahead with uncertainty, compute the cost of waiting, and backtest against what really happened.

Phase 3 — Reason → Serve → Show. Understand the question, apply the human-approval tiers, serve the result as data, and display it in the web app.

Why it exists: so any engineer can trace any number back to the exact line of code that produced it. That traceability is a core part of why the tool can be trusted.

8. The choices we made, and why

A few decisions shaped the whole project. Here they are in plain terms.

Our choice	Why we made it
Calibrate for Los Angeles first	LA has the richest public data, so we could make one city truly accurate before extending to others.
Use a simple, explainable model — not flashy AI	We only have data for a handful of cities; a complex model would “memorize” instead of learn. A simple one is more honest, and we can show exactly what drives its answer.
Always show a range, never a single number	The future is uncertain. A range is honest; a single number would pretend to a precision we don’t have.
Never let the AI invent a number	The AI can explain and phrase, but a built-in check rejects any figure it didn’t get from our own calculations.
A human makes the final budget decision	The tool informs the timing trade-off; it stops before recommending an actual allocation, which needs human judgment and accountability.
Everything runs offline	No data leaves the computer and no internet account is needed — better for privacy, and for a reliable demo.

9. What we deliberately don’t do

To keep the tool trustworthy, we set clear limits. It does not decide budgets, does not predict anything about individual people, and only looks at fairness at the whole-population level.

Where our data is weaker, we label it as lower-confidence and widen the range rather than hide it. One city (New York) works very differently because of its shelter laws, so we point that out instead of pretending our model fits it perfectly.

10. The short version (for talking points)

WaitCost turns free public data into a clear, dollar answer about the cost of waiting to act on homelessness. A small offline AI understands the question and writes the explanation, but trusted code does every calculation and owns every number, and a human makes the final spending decision. The counts come from official HUD reports, the economics from the U.S. Census, and the costs from a published study; we collect and verify them once, then everything runs offline. We chose simple, explainable methods on purpose, we always show a range, and we can trace every figure back to the code that produced it.